

ソフトマックス回帰：分類のための線形ニューラルネットワーク

Dive into Deep Learning 輪読ゼミ

山口 海

May 18, 2026

4.1. ソフトマックス回帰の概要

4.1.1. 回帰から分類へ

4.1.1.1. 線形モデル

4.1.1.2. ソフトマックス関数

4.1.1.3. ベクトル化

4.1.2. 損失関数

4.1.2.1. 対数尤度

4.1.2.2. ソフトマックスと交差エントロピー損失

4.1.3. 情報理論の基礎

4.1.3.1. 情報エントロピー

4.1.3.3. 交差エントロピーの再考

4.2. 画像分類データセットと実践

4.2.1. データセットの読み込み

4.2.2 ミニバッチの読み込み

4.2.3 画像の可視化

4.2.4. まとめ

4.3. 分類の基礎モデル

4.3.1. Classifier クラス

4.3.2. 精度

4.3.3 まとめ

4.1. ソフトマックス回帰の概要

4.1.1. 回帰から分類へ

線形回帰では、入力から連続値を予測した。

例：回帰

家の価格はいくらか？

患者は何日入院するか？

ソフトマックス回帰では、入力からカテゴリを予測する。

例：分類

この画像は猫・犬・鶏のどれか？

このメールは迷惑メールか？

分類では、単に数値を出すだけでなく、
各クラスに属する確率として解釈できる出力がほしい

分類には2つの見方がある

ハードな分類

入力を1つのクラスに割り当てる.

$$\text{image} \mapsto \text{dog}$$

ソフトな分類

各クラスに属する確率を出す.

$$\text{image} \mapsto \begin{cases} P(\text{cat}) = 0.1 \\ P(\text{dog}) = 0.8 \\ P(\text{chicken}) = 0.1 \end{cases}$$

ソフトマックス回帰では, この確率的な出力を作る.

単純な画像分類問題

ここでは、 2×2 のグレースケール画像を考える。

$$\begin{pmatrix} x_1 & x_2 \\ x_3 & x_4 \end{pmatrix} \longmapsto (x_1, x_2, x_3, x_4)$$

- 各画素値を 1 つの特徴量として扱う
- 入力は 4 次元ベクトル

$$\boldsymbol{x} = (x_1, x_2, x_3, x_4)^\top$$

- 出力クラスは 3 つ

$\{\text{cat, chicken, dog}\}$

各画像は、この 3 つのうち **どれか 1 つ** に属すると仮定する。

ラベルの表し方

カテゴリを単に整数で表すこともできる.

$$y \in \{1, 2, 3\}$$

しかし, この表し方だと問題がある.

整数ラベルの注意点

- cat < chicken < dog という意図しない大小関係が生まれる
- 多くの分類問題では, クラス間に自然な順序はない

そこで

画像識別や自然言語処理でよく使われるのが, **one-hot 表現**である.

one-hot エンコーディング

分類では，ラベルをベクトルで表すことが多い．

$$y \in \{(1, 0, 0), (0, 1, 0), (0, 0, 1)\}$$

クラス	one-hot ベクトル
cat	(1, 0, 0)
chicken	(0, 1, 0)
dog	(0, 0, 1)

正解クラスの成分だけが 1，それ以外は 0 になる．

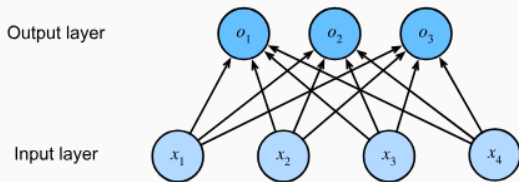
4.1.1.1. 線形モデル：クラスごとにスコアを出す

入力が4次元，クラスが3種類なので，出力も3つ用意する．

$$o_1 = x_1w_{11} + x_2w_{12} + x_3w_{13} + x_4w_{14} + b_1,$$

$$o_2 = x_1w_{21} + x_2w_{22} + x_3w_{23} + x_4w_{24} + b_2,$$

$$o_3 = x_1w_{31} + x_2w_{32} + x_3w_{33} + x_4w_{34} + b_3.$$



- w_{ij} は入力 x_j がクラス i に与える重み
- b_i はクラス i のバイアス項
- o_i はクラス i のスコア

線形モデルの行列表記

先ほどの式は、行列を使うと

$$\begin{pmatrix} o_1 \\ o_2 \\ o_3 \end{pmatrix} = \begin{pmatrix} w_{11} & w_{12} & w_{13} & w_{14} \\ w_{21} & w_{22} & w_{23} & w_{24} \\ w_{31} & w_{32} & w_{33} & w_{34} \end{pmatrix} \begin{pmatrix} x_1 \\ x_2 \\ x_3 \\ x_4 \end{pmatrix} + \begin{pmatrix} b_1 \\ b_2 \\ b_3 \end{pmatrix}$$

と書けるが、 b_i を行列に組み込んで、

$$\begin{pmatrix} o_1 \\ o_2 \\ o_3 \end{pmatrix} = \begin{pmatrix} w_{11} & w_{12} & w_{13} & w_{14} & b_1 \\ w_{21} & w_{22} & w_{23} & w_{24} & b_2 \\ w_{31} & w_{32} & w_{33} & w_{34} & b_3 \end{pmatrix} \begin{pmatrix} x_1 \\ x_2 \\ x_3 \\ x_4 \\ 1 \end{pmatrix}$$

と表記することもある.

なぜそのまま確率にできないのか

線形モデルの出力 o をそのまま確率とみなしたいが、問題がある。

確率として必要な条件

- 各成分が非負である： $\hat{y}_i \geq 0$
- 全クラスの和が1である： $\sum_i \hat{y}_i = 1$

($\hat{y}_i$ はクラス i であると予測する確率)

しかし、線形モデルの出力 o_i は、

$$o_i < 0 \quad \text{や} \quad \sum_i o_i \neq 1$$

になりうる。

そこで、スコアを確率に変換する関数が必要になる。

ソフトマックス関数

確率ベクトルを $\hat{\mathbf{y}}$ と書くと,

$$\hat{\mathbf{y}} = \text{softmax}(\mathbf{o})$$

となり, ソフトマックスは, スコア $\mathbf{o}$ を確率ベクトルに変換する. ここで, $\mathbf{P}(\mathbf{y} = \mathbf{i} \mid \mathbf{x}) \propto \exp(o_i)$ として,

$$\hat{y}_i = \frac{\exp(o_i)}{\sum_j \exp(o_j)}$$

とソフトマックス関数を導入する.

ソフトマックス関数の特徴

- 指数関数により, すべての成分を正にする
- 大きいスコアほど, 大きい確率になる

ソフトマックスのイメージ

例えば、線形モデルの出力が

$$\mathbf{o} = (1.2, 0.3, 2.1)$$

だったとする.

ソフトマックスを通すと,

$$\hat{\mathbf{y}} = \text{softmax}(\mathbf{o}) = (0.26, 0.11, 0.63)$$

のように、確率として解釈できる値になる.

予測クラス

最も確率が大きいクラスを選ぶ.

$$\hat{y}_3 = 0.63 \quad \Rightarrow \quad \text{dog}$$

ソフトマックスは最大クラスを変えない

ソフトマックスはスコアを確率に変換するが、**大小関係**は変えない.

$$\operatorname{argmax}_j \hat{y}_j = \operatorname{argmax}_j o_j$$

つまり、予測クラスだけを知りたいなら、

$$\mathbf{o} = (1.2, 0.3, 2.1)$$

の時点で、最大の成分 o_3 を選べばよい.

重要

ソフトマックスは「どのクラスが最大か」を変えずに、スコアを**確率として解釈できる形**に変換する.

ソフトマックスとボルツマン分布

ソフトマックスは、統計力学に現れるボルツマン分布とよく似た形をしている。

ソフトマックス

$$\hat{y}_i = \frac{\exp(o_i)}{\sum_j \exp(o_j)}$$

ボルツマン分布

$$P(i) = \frac{\exp\left(-\frac{E_i}{k_B T}\right)}{\sum_j \exp\left(-\frac{E_j}{k_B T}\right)}$$

どちらも「指数をとって、総和で割る」ことで確率分布を作っている。以下、 $\beta = 1/(k_B T)$ と置き、 β は逆温度と呼ばれる。

ボルツマン分布の意味

ボルツマン分布では、量子状態 i が出現する確率を $P(i) \propto \exp(-\beta E_i)$ を仮定して、

$$P(i) = \frac{1}{Z} \exp(-\beta E_i)$$

と表す。ここで、分母は規格化定数で、

$$Z = \sum_j \exp(-\beta E_j)$$

であり、分配関数と呼ばれる。

- E_i : 状態 i のエネルギー
- T : 温度
- k_B : ボルツマン定数
- Z : 分配関数

ソフトマックスとの対応関係

ソフトマックスでは、スコア o_i が大きいクラスほど予測確率が大きくなる。

ソフトマックス

$$\hat{y}_i = \frac{\exp(o_i)}{\sum_j \exp(o_j)}$$

ボルツマン分布

$$P(i) = \frac{\exp(-\beta E_i)}{\sum_j \exp(-\beta E_j)}$$

ボルツマン分布と比較すると、

$$o_i \leftrightarrow -\beta E_i$$

と対応していると見なせる。

- ソフトマックス：スコアが高いほど確率が高い
- ボルツマン分布：エネルギーが低いほど確率が高い

分配関数の対応関係

ソフトマックスの分配関数

$$\sum_j \exp(o_j)$$

ボルツマン分布の分配関数

$$Z = \sum_j \exp(-\beta E_j)$$

統計力学において、分配関数 Z を求めることができれば、その系の熱力学のすべてがわかると言われる。

自由エネルギー F

$$F = -\frac{1}{\beta} \ln Z$$

- 分配関数 Z を求めることができれば、これを微分するだけでエントロピーや内部エネルギーなど、その系の熱力学のすべてがわかる。

4.1.1.3. ベクトル化

計算効率のため、1つのデータごとに計算するのではなく、 n 個のミニバッチに対してまとめて計算する。

$$\mathbf{X} \in \mathbb{R}^{n \times d}, \quad \mathbf{W} \in \mathbb{R}^{d \times q}, \quad \mathbf{b} \in \mathbb{R}^{1 \times q}$$

$$\mathbf{O} = \mathbf{XW} + \mathbf{b}, \quad \mathbf{O} \in \mathbb{R}^{n \times q}$$

$$\hat{\mathbf{Y}} = \text{softmax}(\mathbf{O})$$

- n : ミニバッチサイズ
- d : 入力特徴量の数
- q : クラス数

計算の流れ

1. $\mathbf{X}$ をまとめて線形変換
2. クラスごとのスコア $\mathbf{O}$ を出す
3. 各行に softmax をかける

ミニバッチを行列で表す

入力特徴量の数を d ，ミニバッチサイズを n とする． n 個の入力データを縦に並べる．

$$\mathbf{X} = \begin{pmatrix} - & (\mathbf{x}^{(1)})^\top & - \\ & \vdots & \\ - & (\mathbf{x}^{(n)})^\top & - \end{pmatrix} \in \mathbb{R}^{n \times d}$$

$\mathbf{X}$ の各行が，1つの入力データを表す．重み行列 $\mathbf{W} \in \mathbb{R}^{d \times q}$ を掛け，1つのデータに対するクラスごとの補正であるバイアス $\mathbf{b} \in \mathbb{R}^{1 \times q}$ を足すと，

$$\mathbf{O} = \begin{pmatrix} - & (\mathbf{x}^{(1)})^\top \mathbf{W} & - \\ & \vdots & \\ - & (\mathbf{x}^{(n)})^\top \mathbf{W} & - \end{pmatrix} + \begin{pmatrix} - & \mathbf{b} & - \\ & \vdots & \\ - & \mathbf{b} & - \end{pmatrix}$$

スコア行列 $\mathbf{O} \in \mathbb{R}^{n \times q}$ を得る．このような自動的な拡張を**ブロードキャスト**という．

ベクトル化されたソフトマックス

線形モデルで得たスコア行列に対して、softmax をかける。

$$\hat{\mathbf{Y}} = \text{softmax}(\mathbf{O})$$

$$\hat{Y}_{ij} = \frac{\exp(O_{ij})}{\sum_{k=1}^q \exp(O_{ik})}$$

softmax は、各行ごとに計算される。

$$\hat{\mathbf{Y}} = \begin{pmatrix} \hat{y}_1^{(1)} & \hat{y}_2^{(1)} & \cdots & \hat{y}_q^{(1)} \\ \hat{y}_1^{(2)} & \hat{y}_2^{(2)} & \cdots & \hat{y}_q^{(2)} \\ \vdots & \vdots & \ddots & \vdots \\ \hat{y}_1^{(n)} & \hat{y}_2^{(n)} & \cdots & \hat{y}_q^{(n)} \end{pmatrix} \in \mathbb{R}^{n \times q}$$

$\hat{y}_j^{(i)}$ は、 i 番目のデータがクラス j であると予測する確率を表し、各行の和は 1 になる。

4.1.2. 損失関数

ソフトマックス回帰では、モデルの出力を各クラスに属する確率として解釈する。

$$\hat{y}_j = \frac{\exp(o_j)}{\sum_{k=1}^q \exp(o_k)}$$

$$\hat{\mathbf{y}} = (\hat{y}_1, \hat{y}_2, \dots, \hat{y}_q), \quad \hat{y}_j = P(y = j \mid \mathbf{x})$$

損失関数は、正解クラスに高い確率を与えるほど小さくなるように作りたい。

4.1.2. 損失関数

$$\hat{\mathbf{y}} = (\hat{y}_{\text{cat}}, \hat{y}_{\text{dog}}, \hat{y}_{\text{bird}}), \quad \mathbf{y} = (0, 1, 0)$$

ここでは, 正解クラスは犬である.

良い予測

$$\hat{\mathbf{y}} = (0.1, 0.7, 0.2)$$

$$P(y = \text{dog} \mid \mathbf{x}) = 0.7$$

正解クラスである犬に高い確率を与えている.

⇒ 損失は小さい

悪い予測

$$\hat{\mathbf{y}} = (0.8, 0.1, 0.1)$$

$$P(y = \text{dog} \mid \mathbf{x}) = 0.1$$

正解クラスである犬に低い確率しか与えていない.

⇒ 損失は大きい

最尤推定から損失関数を作る

データセットを

$$\{(\mathbf{x}^{(1)}, y^{(1)}), \dots, (\mathbf{x}^{(n)}, y^{(n)})\}$$

とする.

各データが独立にサンプリングされると仮定すると, データセット全体の条件付き確率は

$$P(\mathbf{Y} \mid \mathbf{X}) = \prod_{i=1}^n P(y^{(i)} \mid \mathbf{x}^{(i)})$$

と分解できる.

この量を大きくすることは, 観測された正解ラベルが最も起こりやすくなるようにモデルを調整することに対応する.

負の対数尤度

尤度は積の形なので、そのままでは扱いにくい。そこで対数を取る。

$$\log P(\mathbf{Y} \mid \mathbf{X}) = \log \prod_{i=1}^n P(y^{(i)} \mid \mathbf{x}^{(i)}) = \sum_{i=1}^n \log P(y^{(i)} \mid \mathbf{x}^{(i)})$$

尤度最大化は、負の対数尤度最小化と同値である。

$$-\log P(\mathbf{Y} \mid \mathbf{X}) = \sum_{i=1}^n -\log P(y^{(i)} \mid \mathbf{x}^{(i)})$$

ここで i 番目のデータ 1 つに対する損失を

$$l(\mathbf{y}^{(i)}, \hat{\mathbf{y}}^{(i)}) = -\log P(y^{(i)} \mid \mathbf{x}^{(i)})$$

と定義する。

one-hot ラベルと交差エントロピー

正解が第 k クラスであるとする、one-hot ベクトルは

$$\mathbf{y} = (0, \dots, 0, 1, 0, \dots, 0)$$

となる。このとき

$$-\sum_{j=1}^q y_j \log \hat{y}_j = -\log \hat{y}_k$$

となるので、

$$l(\mathbf{y}, \hat{\mathbf{y}}) = -\sum_{j=1}^q y_j \log \hat{y}_j$$

となる。この損失関数は交差エントロピー損失と呼ばれる。

具体例: 正解クラスの確率と交差エントロピー損失

正解ラベルが第2クラスであるとする.

$$\mathbf{y} = (0, 1, 0)$$

良い予測

$$\hat{\mathbf{y}} = (0.1, 0.7, 0.2)$$

正解クラスの確率: $\hat{y}_2 = 0.7$

このときの損失は

$$l(\mathbf{y}, \hat{\mathbf{y}}) = - \sum_{j=1}^3 y_j \log \hat{y}_j = -(0 \log 0.1 \\ + 1 \log 0.7 + 0 \log 0.2) \approx 0.357$$

⇒ 損失は小さい

悪い予測

$$\hat{\mathbf{y}} = (0.8, 0.1, 0.1)$$

$$\hat{y}_2 = 0.1$$

損失は

$$l(\mathbf{y}, \hat{\mathbf{y}}) = -\log 0.1 \approx 2.303$$

⇒ 損失は大きい

4.1.2.2. ソフトマックスと交差エントロピー損失

ソフトマックス回帰では, ロジット $\mathbf{o}$ を確率に変換する.

$$\hat{y}_j = \text{softmax}(\mathbf{o})_j = \frac{\exp(o_j)}{\sum_{k=1}^q \exp(o_k)}$$

交差エントロピー損失は

$$l(\mathbf{y}, \hat{\mathbf{y}}) = - \sum_{j=1}^q y_j \log \hat{y}_j$$

であった.

ここでは, この2つを組み合わせると何が起こるかを見る. 特に, 勾配が非常に簡単な形になる.

交差エントロピーにソフトマックスを代入する

交差エントロピー損失： $l(\mathbf{y}, \hat{\mathbf{y}}) = -\sum_{j=1}^q y_j \log \hat{y}_j$ に

$\hat{y}_j = \frac{\exp(o_j)}{\sum_{k=1}^q \exp(o_k)}$ を代入して $\sum_{j=1}^q y_j = 1$ を使うと,

$$\begin{aligned} l(\mathbf{y}, \hat{\mathbf{y}}) &= -\sum_{j=1}^q y_j \log \frac{\exp(o_j)}{\sum_{k=1}^q \exp(o_k)} = -\sum_{j=1}^q y_j \left(o_j - \log \sum_{k=1}^q \exp(o_k) \right) \\ &= \sum_{j=1}^q y_j \log \sum_{k=1}^q \exp(o_k) - \sum_{j=1}^q y_j o_j = \log \sum_{k=1}^q \exp(o_k) - \sum_{j=1}^q y_j o_j \end{aligned}$$

となる. 任意のロジット o_j に対して勾配を取る.

$$\begin{aligned} \frac{\partial}{\partial o_j} l(\mathbf{y}, \hat{\mathbf{y}}) &= \frac{\partial}{\partial o_j} \log \sum_{k=1}^q \exp(o_k) - \frac{\partial}{\partial o_j} \sum_{k=1}^q y_k o_k \\ &= \frac{\exp(o_j)}{\sum_{k=1}^q \exp(o_k)} - y_j = \text{softmax}(\mathbf{o})_j - y_j \\ &= \hat{y}_j - y_j \end{aligned}$$

勾配の意味

$$\frac{\partial l}{\partial o_j} = \hat{y}_j - y_j$$

つまり、勾配は「予測確率 - 正解ラベル」である。

例： $\mathbf{y} = (0, 1, 0)$

良い予測

$$\hat{\mathbf{y}} = (0.1, 0.7, 0.2)$$

$$\hat{\mathbf{y}} - \mathbf{y} = (0.1, -0.3, 0.2)$$

正解クラスの修正は小さい。

悪い予測

$$\hat{\mathbf{y}} = (0.8, 0.1, 0.1)$$

$$\hat{\mathbf{y}} - \mathbf{y} = (0.8, -0.9, 0.1)$$

強く修正される。

$$\text{勾配降下法： } \mathbf{o} \leftarrow \mathbf{o} - \eta \frac{\partial l}{\partial \mathbf{o}}$$

正解クラスのスコアを上げ、不正解のスコアを下げるように更新

情報量: 驚きの大きさ

情報理論では、事象が起きたときの驚きの大きさを情報量として考える.

情報量 = 驚きの大きさ

事象 j が起こる確率を $P(j)$ とすると、その情報量を

$$I(j) := -\log P(j)$$

と定義する.

まれな事象は貴重なサンプルとなり情報量は大きい.

確率が低い $\Leftrightarrow$ 情報量が大きい

情報エントロピー：情報量の期待値

分布 P に従って事象 j が起こるとする. 各事象の情報量は

$$I(j) = -\log P(j)$$

である. したがって, 平均的な情報量は期待値として

$$\mathbb{E}_{j \sim P}[I(j)] = \sum_j P(j)I(j)$$

と書ける. これを情報エントロピーという.

$$\begin{aligned} H[P] &= \sum_j P(j)(-\log P(j)) \\ &= -\sum_j P(j) \log P(j) \end{aligned}$$

4.1.3.3. 交差エントロピーの再考

真の分布を P , モデルの予測分布を Q として交差エントロピーを定義する.

$$H(P, Q) := - \sum_j P(j) \log Q(j)$$

分類の交差エントロピー損失は $P(j) = y_j$, $Q(j) = \hat{y}_j$ に対応.

$$H(P, Q) = - \sum_{j=1}^q y_j \log \hat{y}_j = l(\mathbf{y}, \hat{\mathbf{y}})$$

交差エントロピー損失は, 真の分布 P に従って起こる事象を, 予測分布 Q で説明するときの平均的情報量である.

予測分布 Q が真の分布 P に近いほど小さくなる. つまり $Q = P$ のとき, 交差エントロピーは最小になる.

$$H(P, Q) = H(P, P) = H(P)$$

4.2. 画像分類データセットと実践

4.2. 画像分類データセット

この節では、画像分類の実験に用いる **Fashion-MNIST** を準備する.

この節の位置づけ

- モデルの理論ではなく、データの準備を扱う
- 画像とラベルの組を読み込む
- ミニバッチとしてモデルに入力できる形にする

以降のソフトマックス回帰では、このデータを使って分類を行う.

データセット → ミニバッチ → 分類モデル

Fashion-MNIST とは

Fashion-MNIST は、衣類画像を 10 クラスに分類するデータセットである。

特徴

- 学習データ：60000 枚
- テストデータ：10000 枚
- クラス数：10
- 画像サイズ：28 × 28
- 白黒画像なのでチャンネル数は 1

ラベル	クラス名
0	t-shirt
1	trouser
2	pullover
3	dress
4	coat
5	sandal
6	shirt
7	sneaker
8	bag
9	ankle boot

画像 x $\longrightarrow$ ラベル $y \in \{0, 1, \dots, 9\}$

PyTorch でデータセットを読み込む

PyTorch では, `torchvision.datasets` を用いて Fashion-MNIST を読み込める.

```
class FashionMNIST(d2l.DataModule):    #@save
    """The Fashion-MNIST dataset."""
    def __init__(self, batch_size=64, resize=(28, 28)):
        super().__init__()
        self.save_hyperparameters()
        trans = transforms.Compose([transforms.Resize(resize),
                                     transforms.ToTensor()])
        self.train = torchvision.datasets.FashionMNIST(
            root=self.root, train=True, transform=trans, download=
            True)
        self.val = torchvision.datasets.FashionMNIST(
            root=self.root, train=False, transform=trans, download=
            True)
```

- `train=True`: 学習用データを使う
- `train=False`: 評価用データ, 検証用データを使う
- `ToTensor()`: 画像を数値テンソルに変換

テンソルの形の意味

1 枚の画像の形は,

```
data.train[0][0].shape  
  
# torch.Size([1, 32, 32])
```

画像テンソルの形

$$\text{image} \in \mathbb{R}^{c \times h \times w}$$

`[1, 32, 32]` = [channel 1, height 32, width 32]

32×32 の白黒画像なので, チャンネル数は 1 である.

注意

元画像は 28×28 だが, ここでは D2L に合わせて 32×32 にリサイズしている.

4.2.2 ミニバッチの読み込み

学習では、複数枚をまとめたミニバッチで処理する。

```
@d2l.add_to_class(FashionMNIST)  #@save
def get_dataloader(self, train):
    data = self.train if train else self.val
    return torch.utils.data.DataLoader(data, self.batch_size, shuffle
                                       =train,
                                       num_workers=self.num_workers)
X, y = next(iter(data.train_dataloader()))
print(X.shape, X.dtype, y.shape, y.dtype)

# torch.Size([64, 1, 32, 32]) torch.float32 torch.Size([64]) torch.
  int64
```

ミニバッチの形状： $n = 64, c = 1, h = w = 32$

$$\mathbf{X} \in \mathbb{R}^{n \times c \times h \times w}$$

テンソルの形：`[batch size, channel, height, width]`
32 × 32 の白黒画像を 64 枚まとめている。

4.2.3 画像の可視化

データが正しく読み込めているかを確認するために、画像を表示する。

```
@d2l.add_to_class(FashionMNIST)
def visualize(self, batch, nrows=1, ncols=8, labels=[]):
    X, y = batch
    if not labels:
        labels = self.text_labels(y)
    d2l.show_images(X.squeeze(1), nrows, ncols,
                    titles=labels)
```

画像とラベルが対応しているか、リサイズやテンソル変換が意図通りかを確認する。

4.2 節のまとめ

この節の要点

- Fashion-MNIST は 10 クラスの衣類画像分類データセットである.
- PyTorch では `torchvision.datasets.FashionMNIST` で読み込める.
- 画像は [チャンネル数, 高さ, 幅] のテンソルとして扱う.
- ミニバッチでは [バッチサイズ, チャンネル数, 高さ, 幅] になる.
- `DataLoader` は, シャッフル・ミニバッチ化・並列読み込みを担う.

4.3. 分類の基礎モデル

4.3. 分類の基礎モデル

この節でやること

分類問題で何度も使う処理を、共通の部品としてまとめる。

分類モデルでは、毎回ほぼ同じ流れが現れる：

入力 $\mathbf{X}$ $\longrightarrow$ モデル $\longrightarrow$ 予測 $\hat{\mathbf{Y}}$ $\longrightarrow$ 損失・精度の計算

- **損失**：学習のために最小化する量
- **精度**：分類器がどれくらい正解したかを表す指標

$$\text{accuracy} = \frac{\text{正解したデータ数}}{\text{全データ数}}$$

4.3.1. Classifier クラス

分類モデル用の基底クラスを用意する.

```
class Classifier(d2l.Module):    #@save
    """分類モデルの基底クラス。"""
    def validation_step(self, batch):
        Y_hat = self(*batch[:-1])
        self.plot('loss', self.loss(Y_hat, batch[-1]), train=False)
        self.plot('acc', self.accuracy(Y_hat, batch[-1]), train=False)
    )
```

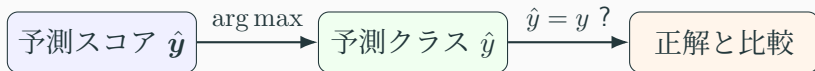
検証データに対して,

予測 → 損失の計算 → 精度の計算

をまとめて行う.

4.3.2. 精度 accuracy

分類では、予測確率またはスコアから、最終的に1つのクラスを選ぶ。



分類精度

$$\text{accuracy} = \frac{\text{正しく分類できたデータ数}}{\text{全データ数}}$$

これを数式で書くと、

$$\text{accuracy} = \frac{1}{n} \sum_{i=1}^n \mathbf{1}(\hat{y}_i = y_i)$$

ここで

$$\mathbf{1}(\hat{y}_i = y_i) = \begin{cases} 1 & (\hat{y}_i = y_i) \\ 0 & (\hat{y}_i \neq y_i) \end{cases}$$

各データについて、最大スコアのクラスを予測クラスとする。

```
@d2l.add_to_class(Classifier)  #@save
def accuracy(self, Y_hat, Y, averaged=True):
    """正しい予測の数を計算する。"""
    Y_hat = d2l.reshape(Y_hat, (-1, Y_hat.shape[-1]))
    preds = d2l.astype(d2l.argmax(Y_hat, axis=1), Y.dtype)
    compare = d2l.astype(preds == d2l.reshape(Y, -1), d2l.float32)
    return d2l.reduce_mean(compare) if averaged else compare
```

流れ

$$\hat{Y} \in \mathbb{R}^{n \times q} \xrightarrow{\arg \max} \hat{y}_1, \dots, \hat{y}_n \xrightarrow{\hat{y}_i = y_i} 0/1$$

- averaged=True: 正解率の平均を返す
- averaged=False: 各データの正誤を 0/1 で返す

4.3 の要点

分類モデルでは、予測結果を評価するために精度 accuracy を計算する.

- Classifier クラスは分類モデルの共通処理をまとめる
- 検証時には、損失 loss と精度 acc を記録する
- 予測クラスは

$$\operatorname{argmax}_j \hat{y}_j$$

で決める

- 精度は

$$\frac{\text{正解数}}{\text{全データ数}}$$

である

次節では、この仕組みを使ってソフトマックス回帰を実装する.